

Dear ImGui Bundle User Interface - Framework Elements

Bachelor Thesis



Author: Constantinos Georgiou (Student ID: 4789)

Supervisor: Prof. George Papagiannakis

Co-Supervisor: Dr. Manos Kamarianakis

Department of Computer Science
School of Sciences & Engineering
University of Crete
Heraklion, Greece

September 20, 2024

Abstract

This thesis focuses on the Elements Framework, which, despite its powerful capabilities, has faced usability challenges due to its size and complexity. One of the key limitations of its current design, which is built upon the Entity Component Scene-graph System (ECSS), is the somewhat cumbersome navigation experience it presents to users which is enhanced in this thesis.

Recognizing the need for a more user-friendly interface, especially for those less familiar with complex frameworks, we set out to enhance Elements with a focus on accessibility and ease of use. The primary objective was to simplify the interface and create a more intuitive environment for all users.

A major milestone in this endeavor was the implementation of a node editor. This visual interface provides a clear, structured view of the framework's entities, components, and scene relationships. By representing these complex interactions visually, the node editor greatly improves navigation and understanding, offering users a more transparent view of the system's inner workings.

One of the core areas targeted for improvement was the manipulation of entity transformations, an essential feature in any framework of this nature. We recognized the need to streamline this process and make it more efficient.

To meet this goal, we integrated transformation gizmos into the framework. These interactive tools allow users to easily translate, rotate, and scale entities along the X, Y, and Z axes, providing a more intuitive and precise approach to entity manipulation. In addition, to ensure seamless and accurate control over the camera system, a dedicated camera gizmo was also introduced. These improvements significantly enhance the user experience, bringing Elements closer to the level of usability found in modern game engines.

Declaration of Authorship

I, Constantinos Georgiou, declare that this thesis titled, "Dear ImGui Bundle User Interface - Framework Elements", and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signature: Constantinos Georgiou

Date: September 20, 2024

Acknowledgements

I would like to express my deepest gratitude to my supervising professor, Prof. George Papagiannakis, for his unwavering support and invaluable guidance throughout the course of this project. His insights and direction have been essential to the successful completion of this work.

I would also like to extend my sincere thanks to Dr. Manos Kamarianakis and John Petropoulos for their continuous support and encouragement during the development of this thesis. Their assistance has been greatly appreciated.

Contents

1	Introduction	1
1.1	Computer Graphics	1
1.2	Computer Graphics Frameworks	2
1.3	Elements	3
1.4	Modern Game Engines	4
2	Object Manipulation, Camera Control, and Scenegraph Representation in Elements	5
2.1	Object Manipulation in Elements	5
2.2	Camera Manipulation in Elements	5
2.3	Scenegraph Representation in Elements	5
2.4	Entity Management in Elements	7
2.5	Proposed Enhancements	8
3	Integrating and extending existing frameworks to enhance Elements	10
3.1	ImGui Bundle	10
3.2	ImGuizmo	11
3.3	ImGui Node Editor	12
4	Features	13
4.1	Open GL Framebuffer	13
4.1.1	Inner Workings	13
4.2	Dockspace	14
4.2.1	Description	14
4.3	Gizmos	15
4.3.1	Elements Previous Manipulation Techniques	15
4.3.2	Transformation Gizmos	16
4.3.3	Camera Gizmos	16
4.3.4	Inner Workings	17
4.4	ImGui Node Editor	18
4.4.1	Inner Workings	19
5	Additional Features	22
5.1	Window Menu Bar	22
5.1.1	Description	22
5.1.2	Inner Workings	22
5.2	Entity Management	23
5.2.1	Inner Workings	23

5.3	Scene Window Management	24
5.3.1	Inner Workings	25
6	Known Bugs	27
6.1	Camera Gizmo	27
6.2	Transformation Gizmo Alignment	27
7	Example	28
7.1	Entity Addition	28
7.2	Entity Manipulation	29
7.3	Entity Re-Parenting	32
7.4	Entity Deletion	33
8	Conclusion	34
	References	35

List of Figures

1	The Utah Teapot	1
2	Elements Example - Earth with Shaders	3
3	Unity Transformation Gizmos	4
4	Unity Camera Gizmo	4
5	Elements Transformation Sliders	5
6	Elements Look-At Camera System	6
7	Elements ECSS Node Tree	6
8	Scale Gizmo - Rotation Gizmo - Translation Gizmo	8
9	Proposed Menu Bar	8
10	Example of node editor in a scene consisting of a root entity and a cube entity name node4 with its children components	9
11	Examples made using Dear-Imgui	10
12	Transformation Gizmos Offered by ImGuizmo	11
13	Camera Gizmo (top right) offered by ImGuizmo	11
14	Node Editor Examples	12
15	Framebuffer Loop Procedure	13
16	Attributes of Framebuffer	14
17	Framebuffer with Wireframe off (left) and on (right)	15
18	First Example of Dockable windows	15
19	Old Manipulation system	16
20	Camera Input System	16
21	Transformation Gizmos	16
22	Camera Gizmo (highlighted with red border)	17
23	Loop Process of Transformation gizmos	18
24	Loop Process of Camera gizmos	18
25	Node Editor - First stage(left) and final stage with color cod- ing(right).	21
26	Extended View of Menu Bar with Entity Addition Window Open	22
27	Window Menu Bar utilized in an example of entity addition . . .	23
28	Entity Addition and Removal Windows	24
29	Example - Gizmo does not follow the object	27
30	Elements Main Screen	28
31	Entity Addition	29
32	New entity not visible	29
33	Entity Translation	30
34	Entity Rotation and Scaling	31
35	Entity Re-Parenting	32

36	Entity Removal	33
37	New Version of Elements	34

1 Introduction

1.1 Computer Graphics

Computer graphics is at the heart of numerous technological advancements, driving innovation across a wide array of industries, from entertainment and gaming to scientific research and data visualization. What began in the 1960s with simple wireframe models and rudimentary 2D graphics has evolved into a sophisticated field that powers some of the most immersive and visually compelling experiences in today's digital world. The evolution of computer graphics has been marked by significant milestones, such as the development of raster graphics, the introduction of 3D modeling, and the advent of real-time rendering techniques. These advancements have enabled the creation of incredibly detailed and realistic visual content that is not only essential for the entertainment industry—bringing to life animated films, video games, and virtual reality—but also critical in fields like medicine, where 3D imaging aids in diagnostics and surgery planning, and in scientific research, where complex simulations can be visualized to better understand phenomena. Today, the ability to generate, manipulate, and render visual content with high precision and realism is more crucial than ever, as digital experiences increasingly define how we interact with information and each other in our rapidly advancing technological landscape.

One of the most recognizable model in computer graphics is the Utah or Newell teapot (See Figure 1). Created by Martin Newell in 1975, is a simple 3D model that became a standard reference object in computer graphics. It played a pivotal role in the development of rendering techniques, serving as a common benchmark for testing and demonstrating new algorithms. Its iconic status endures, symbolizing the early challenges and progress in the field of computer graphics.

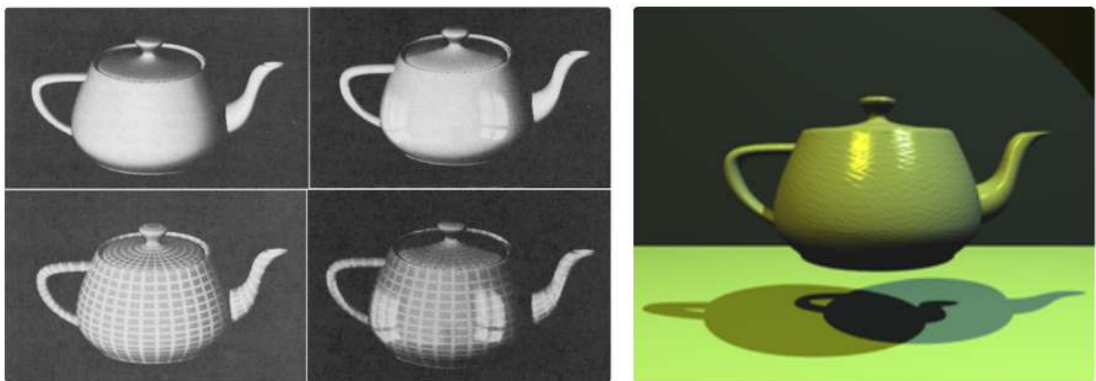


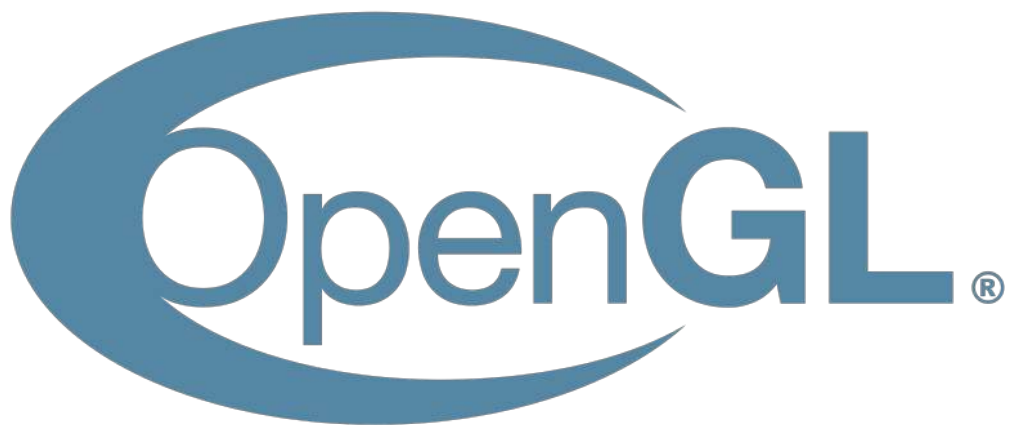
Figure 1: The Utah Teapot

1.2 Computer Graphics Frameworks

As the complexity of visual applications grows, so does the demand for advanced graphics frameworks that can deliver real-time performance, scalable solutions, and seamless integration across various platforms.

Computer graphics frameworks serve as the foundation upon which developers build sophisticated visual applications, enabling the efficient implementation of complex rendering techniques and the management of resources across diverse hardware environments. They abstract the intricacies of low-level graphics programming, offering modular and extensible architectures that facilitate innovation while reducing development time. By leveraging these frameworks, developers can focus on creating unique visual experiences, confident that the underlying systems will ensure optimal performance and quality. As such, the evolution of computer graphics frameworks is integral to the progress of the entire field, continuously pushing the boundaries of what is visually achievable.

OpenGL [1], which will be present throughout this thesis, is such a framework. Open Graphics Library, as its formally known, is a widely-used, cross-platform API for rendering 2D and 3D vector graphics. It provides a robust set of functions that allow developers to interact directly with a system's graphics hardware, enabling the creation of high-performance, visually-rich applications. OpenGL serves as the foundation for many modern graphical interfaces, supporting real-time rendering in everything from video games to scientific visualizations. By offering an abstraction layer for accessing the GPU, OpenGL enables efficient rendering processes, making it a key tool in computer graphics and visualization frameworks.



1.3 Elements

Following the increasing need of computer graphics frameworks, Elements [4], a Computational Entity-component-system in a Scene-graph Pythonic Framework, was developed.

This lightweight, open-source framework¹ designed specifically for computational science and computer graphics (CG) education. It uniquely combines the strengths of an Entity-Component-System (ECS) architecture with the rapid prototyping ease of a Scenegrph-based, Pythonic framework. This innovative approach enhances the teaching of CG concepts, enabling students to explore complex topics such as scenegrphs (heterogeneous directed acyclic graphs), animation, skinning, geometric algebra, and shader-based rendering. Furthermore, Elements extends these concepts to modern applications like 3D scientific visualization.

By integrating ECS within a Scenegrph system, Elements bridges the gap between traditional CG curricula and modern game engines (MGEs), which often employ these advanced techniques but obscure them behind a black-box interface. This framework is designed with an extensible, open-source philosophy, leveraging software design patterns to support a wide range of educational needs. Unlike many existing tools, Elements offers a modern, shader-based pipeline (as opposed to the outdated fixed-function OpenGL). Crucially, it demystifies the CG pipeline, providing students with unique access to challenging scientific, visual, and neural computing concepts in a transparent and accessible manner.

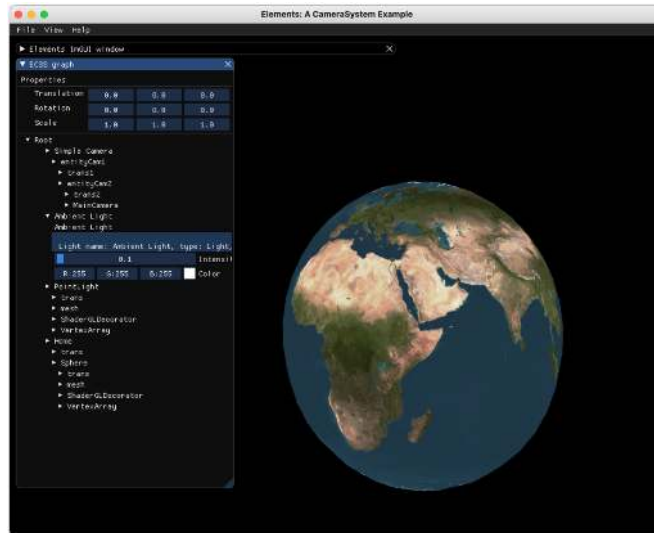


Figure 2: Elements Example - Earth with Shaders

In Figure 2, the earth is simulated by a sphere textured with an image on top and lit by Blinn-Phong Illumination Algorithm.

¹<https://github.com/papagiannakis/Elements>

1.4 Modern Game Engines

Modern game engines have revolutionized the way developers create and manipulate objects within virtual environments, making the process more intuitive and accessible. Engines such as Unity², provide developing tools like gizmos (*See Figure 3*), which are interactive, on-screen controls that allow users to easily move, rotate, and scale objects in real time. Gizmos enable precise positional adjustments of objects without requiring complex coding, making it simpler to position and modify objects directly within the scene. This visual, drag-and-drop approach significantly streamlines the development process, allowing creators to focus more on design and gameplay, rather than the technical details of object manipulation. There is also a camera gizmo (*See Figure 4*), which makes the camera movement and view manipulation that much easier. As a result, modern game engines empower developers of all skill levels to bring their ideas to life with greater efficiency and creativity.

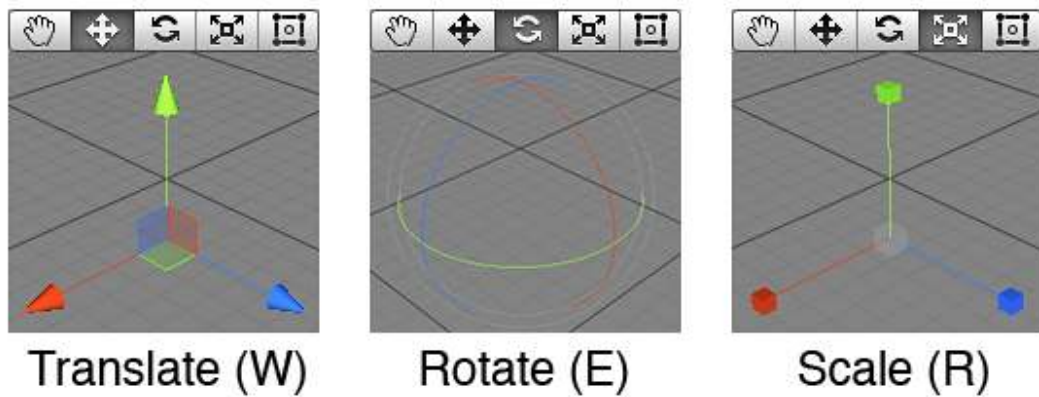


Figure 3: Unity Transformation Gizmos

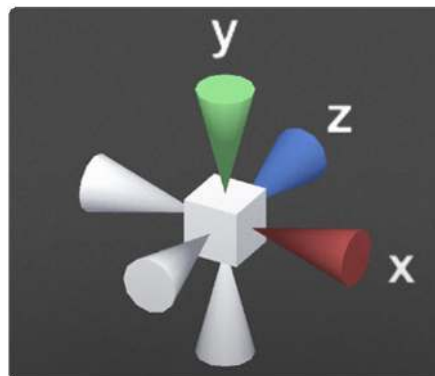


Figure 4: Unity Camera Gizmo

²<https://unity.com/>

2 Object Manipulation, Camera Control, and Scene-graph Representation in Elements

2.1 Object Manipulation in Elements

Having explored how modern game engines simplify object and view manipulation through intuitive tools like gizmos, it's worth comparing these capabilities to those offered by Elements. In Elements, developers manipulate objects using a three-slider interface that controls Translation, Rotation, and Scale along the X, Y, and Z axes (*See Figure 5*). While this approach provides precise control, it can feel cumbersome for some developers, particularly because the movement is not directly mapped to the direction in which the object is moved. This limitation becomes especially noticeable when rotating objects, as the lack of direct manipulation can make it challenging to achieve the desired orientation quickly.

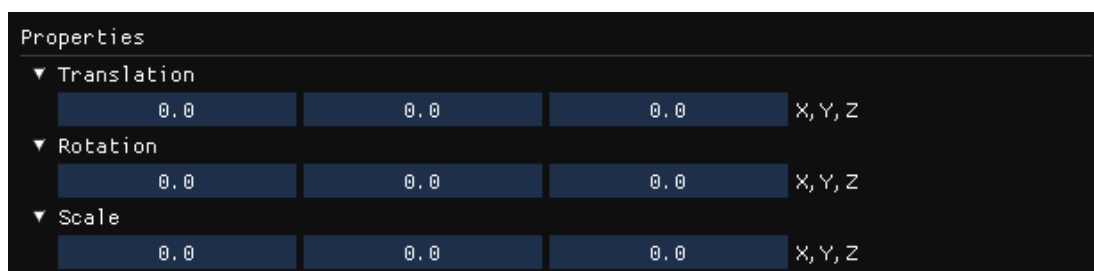


Figure 5: Elements Transformation Sliders

2.2 Camera Manipulation in Elements

The camera system in Elements presents a similar situation. By default, camera movement is managed through simple mouse interactions, such as scrolling and dragging the cursor within the window space. This method works effectively with the standard eye-target-up system (*See Figure 6*), offering an intuitive way to navigate the scene. However, when a dedicated camera system is employed, the camera itself becomes an entity, meaning its movement is also governed by the transformation sliders. This reliance on sliders for camera control can make the process less fluid, especially in comparison to the more interactive camera controls available in modern game engines.

2.3 Scenegraph Representation in Elements

In addition to the challenges mentioned earlier, Elements faces another limitation due to its use of the Entity-Component Scenegraph System. The framework's way to visually display and manage the various entities and components within

2.500	2.500	2.500	Eye
0.000	0.000	0.000	Target
0.000	1.000	0.000	Up

Figure 6: Elements Look-At Camera System

a scene, may be considered in-efficient. Currently, Elements addresses this by traversing the system and presenting all entities and their components in a hierarchical, tree-node structure (*See Figure 7*), starting from the Root Entity and descending through the scenegraph. While this approach keeps the display organized, it introduces complexity, making it cumbersome for developers to navigate the scenegraph and does not offer manipulation of it in any way.

To interact with or modify a specific entity or component, the developer must select it from this tree structure. However, the only visual indication of a selected entity is the corresponding node, which remains open. This can become problematic when multiple nodes are expanded simultaneously, as the system lacks the ability to distinguish which entity should be manipulated. This ambiguity can lead to confusion and errors, particularly in complex scenes where precise control is essential.



Figure 7: Elements ECSS Node Tree

2.4 Entity Management in Elements

Lastly, Elements currently lacks a direct method for adding or removing entities through its graphical user interface (GUI). To perform these actions, developers must engage in low-level coding, which involves editing the underlying code of the example to incorporate a new entity or remove an existing one. This approach can be cumbersome and counterproductive, especially for those seeking a more versatile and user-friendly framework. By requiring such low-level manipulation for tasks that are standard in modern game engines, Elements risks alienating users who expect more intuitive and accessible features.

2.5 Proposed Enhancements

After identifying the challenges that new users might face when working with Elements, we began exploring ways to make the framework more accessible. To simplify entity manipulation, we decided to integrate a gizmo system (*See Figure 8*), similar to that used in Unity. This addition will allow for more direct interaction with objects, making it easier for developers to understand and control the type and direction of manipulation, thereby improving usability and reducing the learning curve.

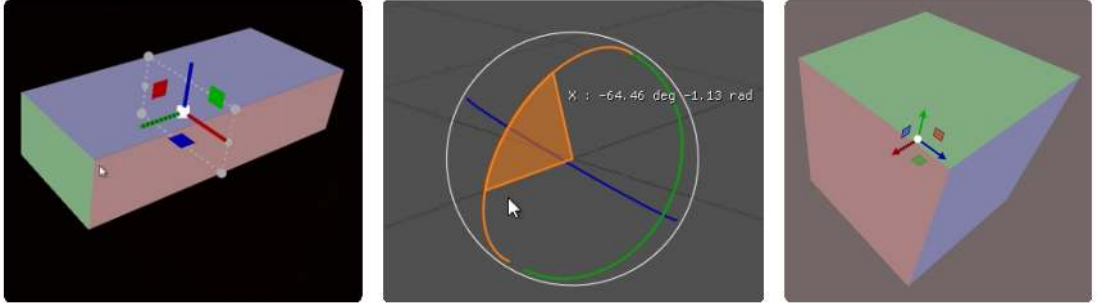


Figure 8: Scale Gizmo - Rotation Gizmo - Translation Gizmo

To further simplify the process of adding or removing entities, we have decided to introduce a main menu bar (*See Figure 9*) at the top of the window. This menu will allow developers to seamlessly create new entities by selecting from one of six predetermined shapes. In addition, developers will have the option to name the entity and assign it to a parent within the scenegraph, streamlining the organizational process. For entity removal, the menu will offer a straightforward method to delete any selected entity, with the exception of the root entity, which serves as the foundation for each scene and cannot be removed. To prevent accidental deletions, a confirmation dialog will appear, ensuring that the developer explicitly approves the removal before it is executed. This feature will enhance Elements by making it more versatile and user-friendly, aligning it with the capabilities found in modern game engines.

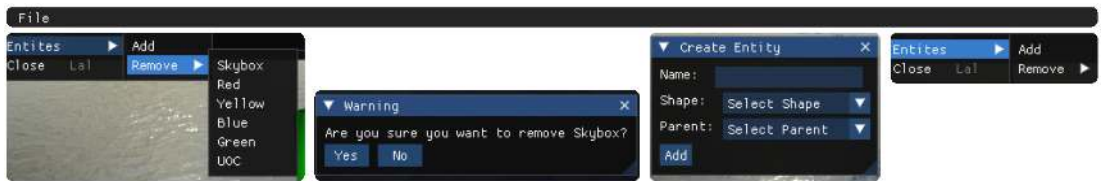


Figure 9: Proposed Menu Bar

To address the complexity of the tree node system, we also opted to incorporate a node editor graph (*See Figure 10* for example) for visualizing the Entity-Component Scenegraph System (ECSS). This graphical representation will com-

plement the existing tree node structure, ensuring that the scenegraph remains organized and hierarchical while enhancing its clarity. The node editor will allow developers to easily select and manipulate entities or components, providing a more intuitive understanding of the scenegraph's architecture. Additionally, to keep the developer informed about the selected entity, the corresponding tree node will automatically open upon selection, and the editor will zoom in on the relevant node within the scenegraph. This approach ensures that developers can maintain focus on their work while navigating the scene with greater ease and precision.

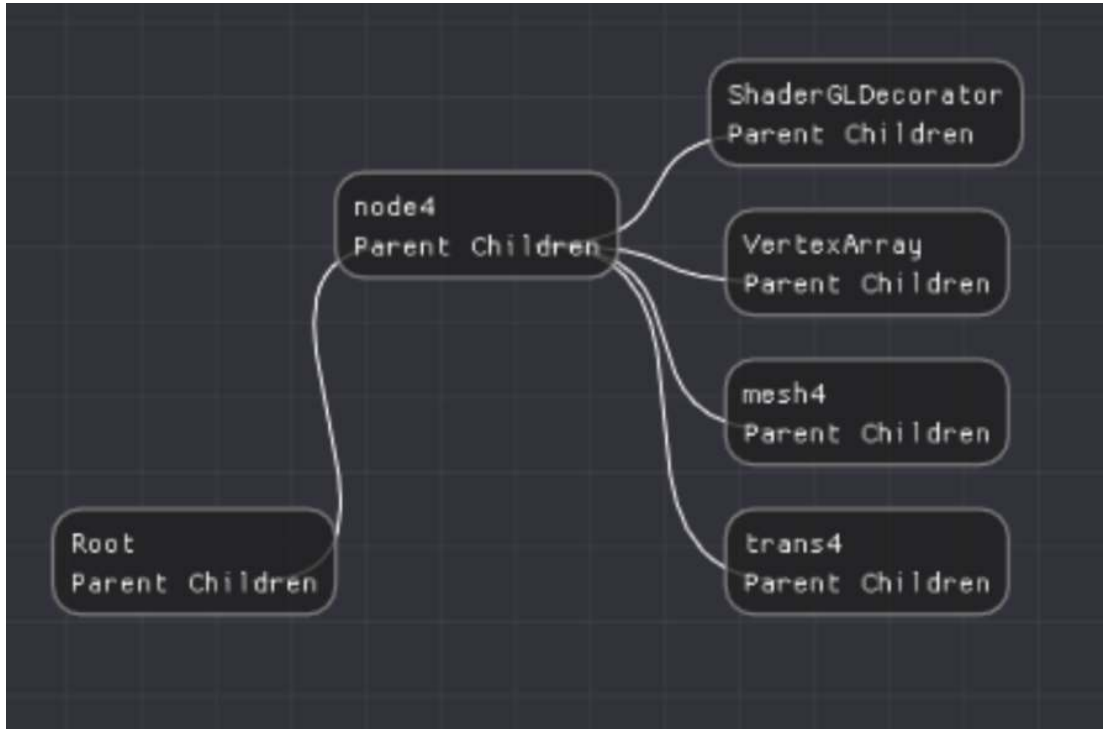


Figure 10: Example of node editor in a scene consisting of a root entity and a cube entity name node4 with its children components

In this thesis, we present a new and updated version of Elements, incorporating features designed to enhance its flexibility, efficiency. These improvements aim to enhance user-friendliness towards developers, allowing them to perform basic operations without needing to engage in complex coding.

3 Integrating and extending existing frameworks to enhance Elements

3.1 ImGui Bundle

For the implementation of the user interface, Elements previously utilized a package called pyimgui[6]. This was a scaled-down, more basic version of Dear ImGui[3], which is designed for C++. To enhance our framework, we sought to incorporate features from the full version of ImGui. Thus, we opted to use the imgui-bundle[7], which encompasses all the additional packages and functionalities that we needed. All available features are documented in an interactive manual³ that is currently accessible online. Notably, this package includes examples of projects that closely resemble modern game engines, showcasing the features we aimed to integrate into Elements (*See Figure 11*).

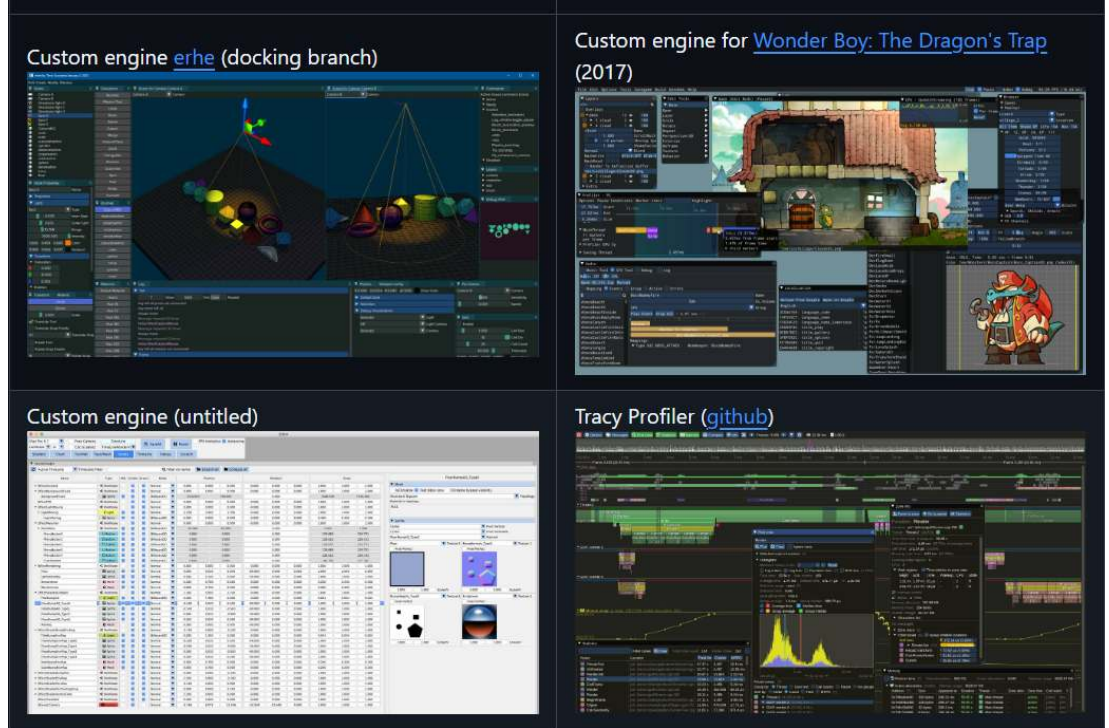


Figure 11: Examples made using Dear-ImGui

³https://traineq.org/ImGuiBundle/emscripten/bin/demo_imgui_bundle.html

3.2 ImGuizmo

As part of the ImGui-Bundle [7], we chose to base our new gizmo system on ImGuizmo [5], which offers a comprehensive set of transformation tools. These include essential gizmos for translation, rotation, and scaling, providing developers with an intuitive and efficient way to manipulate objects within the framework (See Figure 12).



Figure 12: Transformation Gizmos Offered by ImGuizmo

Additionally, a camera gizmo (See Figure 22) is provided to allow developers to easily manipulate the camera, regardless of whether a dedicated camera system is present in the example. This feature ensures intuitive and direct control over camera positioning and movement, enhancing usability and simplifying scene navigation within the framework.



Figure 13: Camera Gizmo (top right) offered by ImGuizmo

3.3 ImGui Node Editor

Also part of the ImGui-Bundle [7], ImGui-Node-Editor [8] introduces a scene-like workspace within an ImGui window, allowing users to add, remove, and move nodes and links, thereby constructing a fully functional node editor. The ImGui-Node-Editor [8] package also supports theme customization, allowing for a degree of visual tailoring within the node editor (*See Figure 14*). This capability aligned perfectly with our vision for Elements, leading us to integrate ImGui-Node-Editor [8] to provide a visual representation of the ECSS System, streamlining the user experience and enhancing scene management within the framework. Due to limitations in Python, we were unable to fully leverage the appearance customization feature. Despite these constraints, we experimented with colorizing the nodes to differentiate them based on their types, offering a visual distinction that aids in scene organization and component identification. (Note that, the node editor that is displayed has no real connection with any back-end functionality and its mainly visual.)

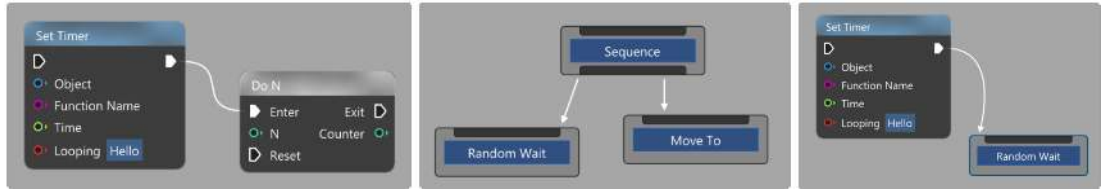


Figure 14: Node Editor Examples

4 Features

4.1 Open GL Framebuffer

From the outset of this thesis, we recognized that the gizmos required a wrapper around an ImGui window to function properly. This presented our first challenge, as Elements renders textures within an SDL [2] window. To address this issue, we decided to utilize OpenGL's [1] framebuffer. By doing so, we could effectively pipe the textures into an ImGui window, facilitating the integration of the gizmos into the existing framework.

4.1.1 Inner Workings

The main module is **Elements/pyGLV/GL/Framebuffer.py**

The procedure for a functioning framebuffer is as follows:

At beginning of runtime, a framebuffer object is created by binding it into the window space, and with it bound, a texture and a renderbuffer are created, one for holding the texture we want drawn in the window and the other for holding a depth component due to us wanting to have a 3D image turned into 2D (*See Figure 15*)

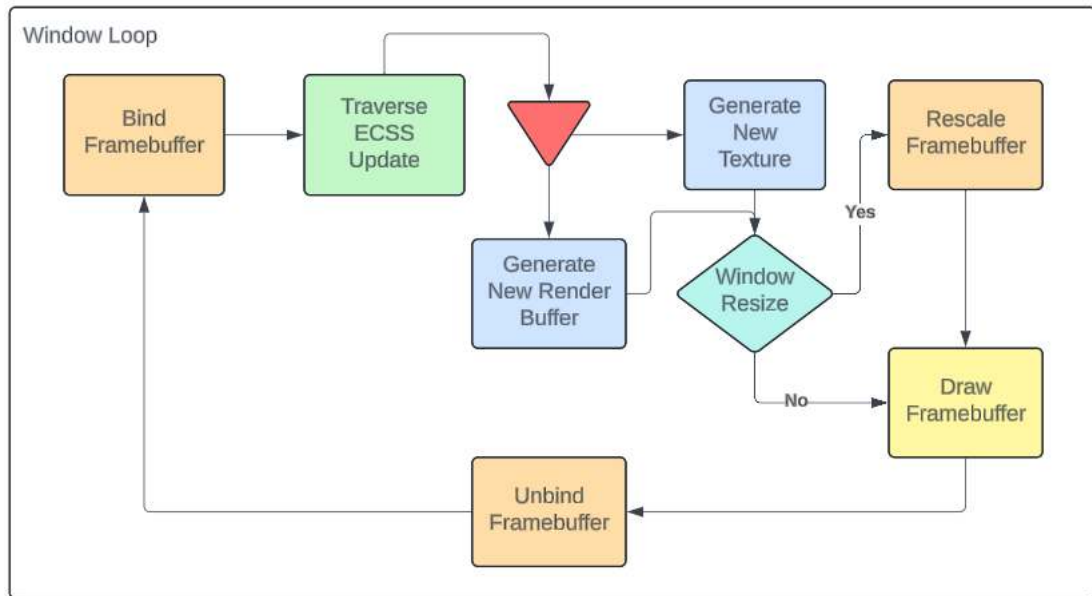


Figure 15: Framebuffer Loop Procedure

It is a singleton class, that consists of (*See Figure 16*):

- A Framebuffer Object (FBO for short)
- A texture which is traversed by ID

- A Renderbuffer Object (RBO for short) that is used for determining the depth that is to be rendered.

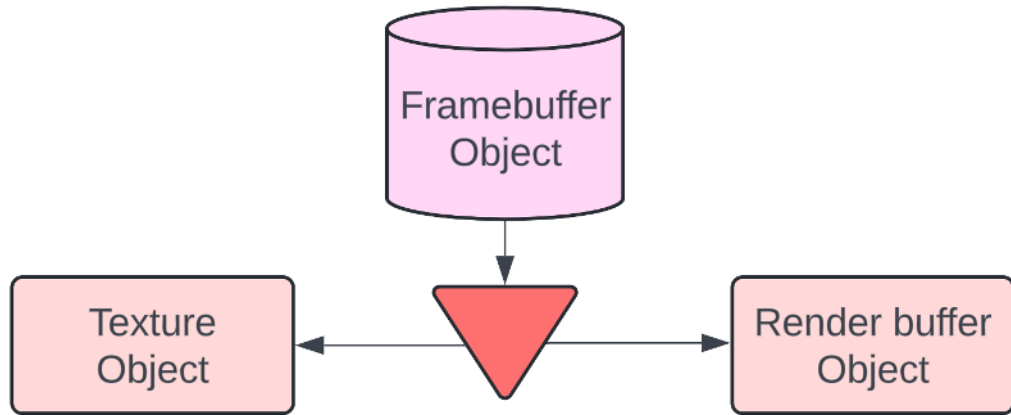


Figure 16: Attributes of Framebuffer

and implements the following methods:

- `createFramebuffer()`: creates and initializes the framebuffer object
- `generateTexture()`: takes the texture currently rendered and generates an ID that the texture will be bound with
- `generateRenderbuffers()`: creates and initializes the renderbuffer object with parameters denoting it as a depth buffer
- `bind/unbindFramebuffer()`: binding of the framebuffer
- `rescaleFramebufffer(width, height)`: rescales the fbo and rbo according to the given dimensions
- `drawFramebuffer(wireframe)`: main loop function that draws the framebuffer, keeping into account to display only the wireframe if true is passed as a parameter, by default it is set to false (*See Figure 17*)

4.2 Dockspace

4.2.1 Description

With the Framebuffer working we realized that it may be cumbersome for the user to shuffle between the different windows and thought that it will be better if there is a way to ensure full visibility of all the windows at all times

ImGui [7] has a dockspace feature so we decided to use that. It keeps the windows

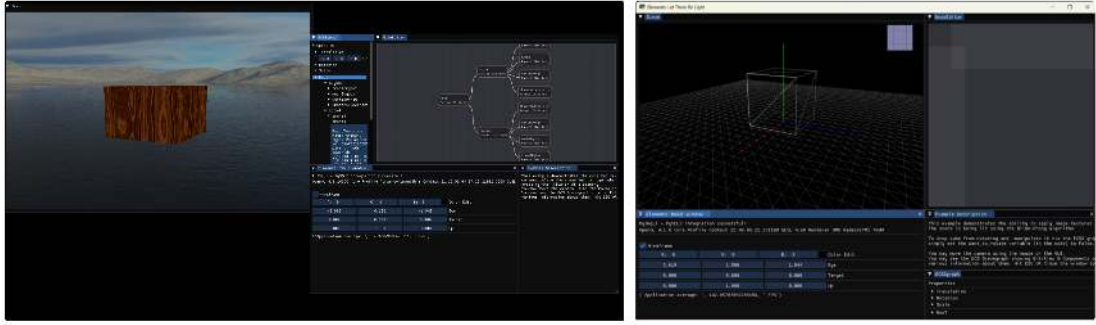


Figure 17: Framebuffer with Wireframe off (left) and on (right)

docked in a certain configuration, and also gives the user the ability to switch said configuration to his liking, giving us a fully versatile app.

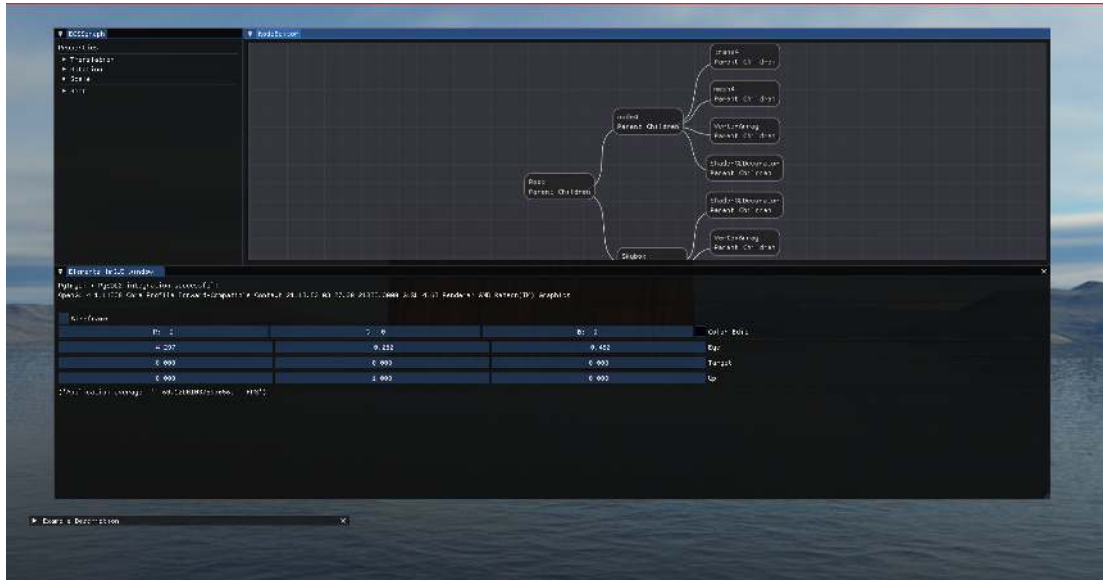


Figure 18: First Example of Dockable windows

4.3 Gizmos

4.3.1 Elements Previous Manipulation Techniques

Manipulating entities within Elements was a cumbersome process. Our aim was to streamline this procedure to improve usability for developers, and to encourage broader adoption of the framework. Previously, entity selection was handled through the ECSS Graph, and manipulation was limited to an inputfloat3 system (See Figure 19), which restricted flexibility and ease of use.

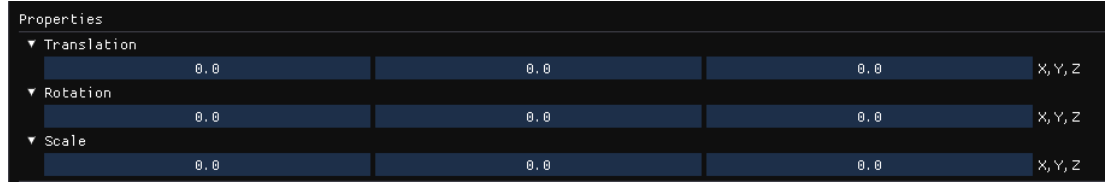


Figure 19: Old Manipulation system

Additionally, we recognized that the only way to manipulate the scene's view was by either dragging with the mouse inside the scene window or using the eye, target, and up `inputfloat3` controls (See Figure 20). This method was functional only when the example did not incorporate a dedicated camera system, limiting its flexibility in more complex scenarios.

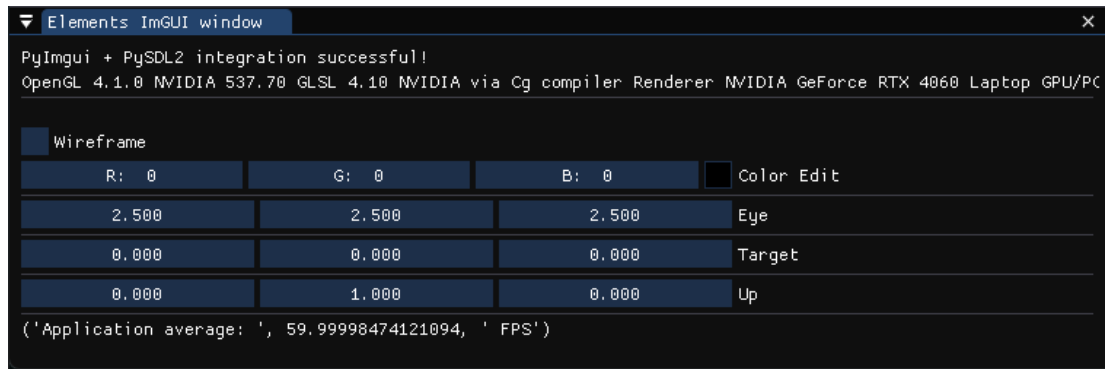


Figure 20: Camera Input System

4.3.2 Transformation Gizmos

To simplify entity manipulation, we aimed to integrate transformation gizmos (See Figure 21) into the framework. This addition significantly enhanced the developer experience, making the framework more intuitive and user-friendly, particularly for new and inexperienced developers.

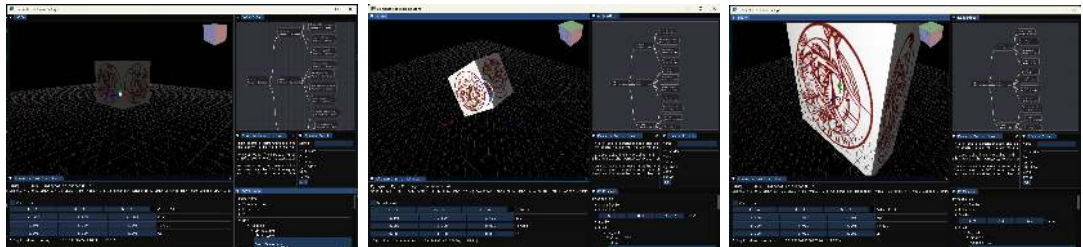


Figure 21: Transformation Gizmos

4.3.3 Camera Gizmos

We additionally leveraged a camera gizmo (See Figure 22), which provided a more direct and intuitive approach to view manipulation, making it easier for

developers to understand and control the scene's perspective.

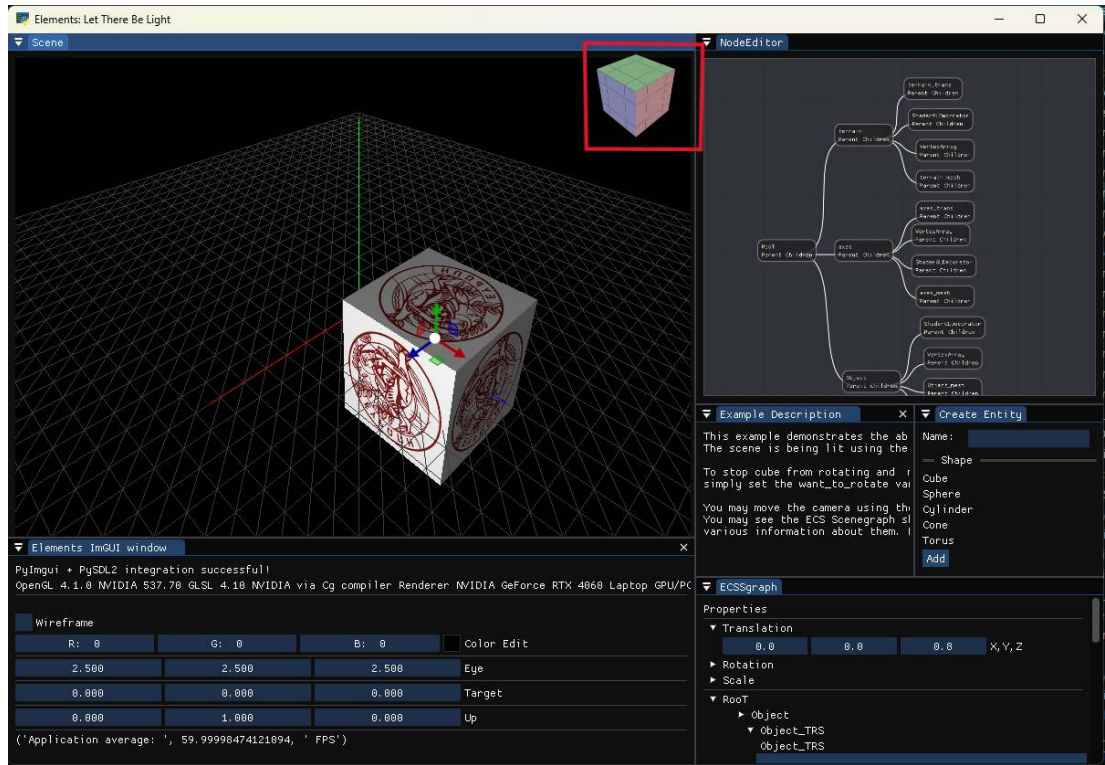


Figure 22: Camera Gizmo (highlighted with red border)

4.3.4 Inner Workings

All inner workings for the gizmos are concentrated inside a single module, **pyGLV/GUI/Gizmos.py**. This module consists of the following attributes:

- **currentGizmoOperation**: holds the current operation being executed e.g. translation, rotation and scale
- **view**: holds the view matrix for the camera manipulation
- **cameraSystem**: in the case of a dedicated camera system existing, this holds a reference to its instance, for manipulating the view from there

and the following methods:

- **setView**: sets the view matrix when the gizmo is not in use, this ensures that the gizmo stays in sync with the rest of the camera operations
- **drawTransformationGizmo(comp)**: takes the component given as a parameter, and if it is of instance **BasicTransform**, it takes the trs and draws the gizmo according to it and the **currentGizmoOperation** attribute

- `drawCameraGizmo()`: draws the camera gizmo according to the view matrix held in the view attribute
- `decompose_look_at()`: decomposes the result of the camera gizmo to get the eye vector for the use of the look at system

The diagram below (*See Figure 23*) illustrates how the transformation gizmos operate during loops.

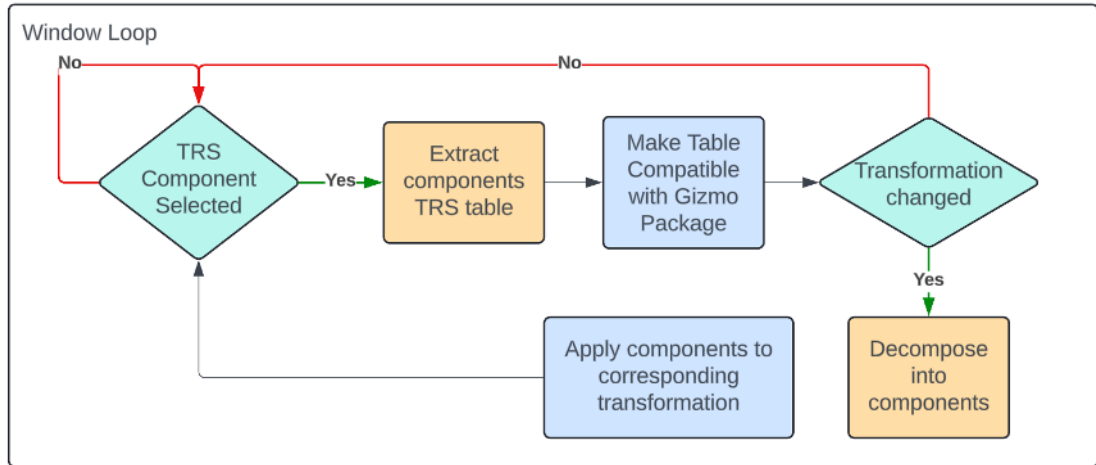


Figure 23: Loop Process of Transformation gizmos

The diagram below (*See Figure 24*) illustrates how the camera gizmo operates during loops.

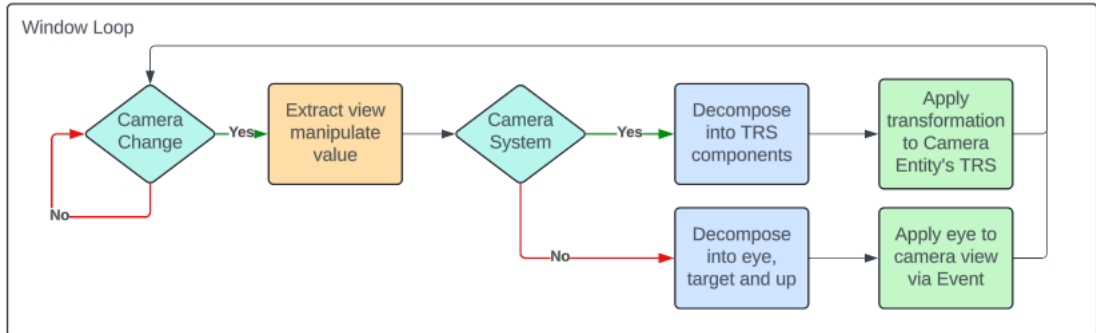


Figure 24: Loop Process of Camera gizmos

4.4 ImGui Node Editor

Elements is a powerful framework that can withstand a great amount of entities in its ECSS System. With that however, comes complexity in understanding the hierarchy of the system due to it being mostly hidden, and viewable only through a tree node menu.

A simple and effective way of visualizing such systems has always been node

diagrams, and that is the way we decided to follow. We additionally opted the distinguish the various types of entities from each other, and proceeded to color code the nodes (*See Figure 25*).

4.4.1 Inner Workings

The main module is **Elements/pyGLV/GUI/NodeEditor.py**

It houses the following data types:

- **IdProvider**: Provides all the id for the nodes and the links that make up the graph . It consists of the following methods:
 - **nextId()**: Provides the next available id
 - **reset()**: Resets the id holder, beginning from 1
- **Node**: Object class for the nodes of the graph and consists of the following attributes:
 - **id**: The id of the node provided by the IdProvider (4.4.1).
 - **childrenPinId**: Holds the id of the children pin
 - **parentPinId**: Holds the id of the parent pin
 - **parent**: Holds a reference to the instance of the current node's parent node
 - **type**: Holds the type of the node e.g. RootEntity, Entity or Component

and the method:

- **display()**: Draws the node onto the editor, handling its color according to its type
- **LinkInfo**: Object class for the links connecting the nodes. It consists of the following attributes:
 - **id**: holds the link's id
 - **input_id**: holds the id of the input end
 - **output_id**: holds the id of the output end
- **NodeEditor**: The main editor class. It consists of the following attributes:
 - **wrapee**: holds a reference to the window wrapee

- **config**: holds the configuration of the current editor
- **editor_context**: holds the editor's context
- **links**: list of links that are present in the graph
- **nodes**: list of nodes that are present in the graph
- **selected**: used for holding a selected item for finding in the ECSS System
- **name**: holds the name of a to be added node when a new entity is added
- **selected_parent**: holds temporarily a node to be used as parent when re-parent happens
- **highlighted**: holds a reference to the highlighted node for focusing when clicked

and the following methods:

- **addNode(comp)**: adds a new node to the graph according to the component given as a parameter. If none, the attributes **selected_parent** and **name** are used.
- **remove(comp)**: removes the node and all links corresponding to the component given as a parameter
- **findNodeByName(name)**: returns a reference to the node that its name is equal to the parameter. If not found, None is returned
- **createLink(parent, child)**: creates a link according to the parent and child link ids given as parameters and appends it to the link list
- **display_nodes()**: calls the **display()** function for all nodes stored in the node list
- **generate()**: Generates the graph on the first frame or after any other tampering has happened e.g. addition of new entity
- **createMode()**: performs all necessary actions needed for the creation
- **on_frame()**: main loop method of the editor

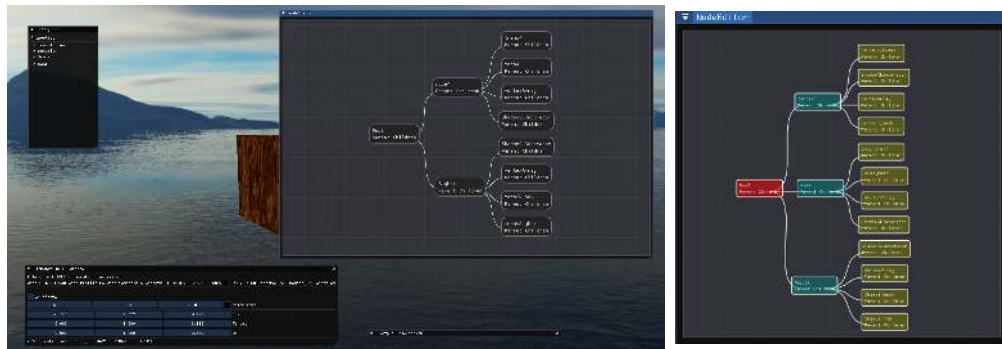


Figure 25: Node Editor - First stage(left) and final stage with color coding(right).

5 Additional Features

5.1 Window Menu Bar

5.1.1 Description

As the new features were integrated, we recognized the potential risk of introducing additional complexities. The extensive range of functionalities we aimed to implement could potentially compromise both ease of use and overall clarity for users.

An effective approach to ensure easy navigation between the various features is the introduction of a simple menu bar (See *Figure 26*) at the top of the window. This menu bar allows users to access essential functionalities related to entity management (See *Section 5.2*), such as adding or removing entities, in a more streamlined and intuitive manner (See *Section 27*).

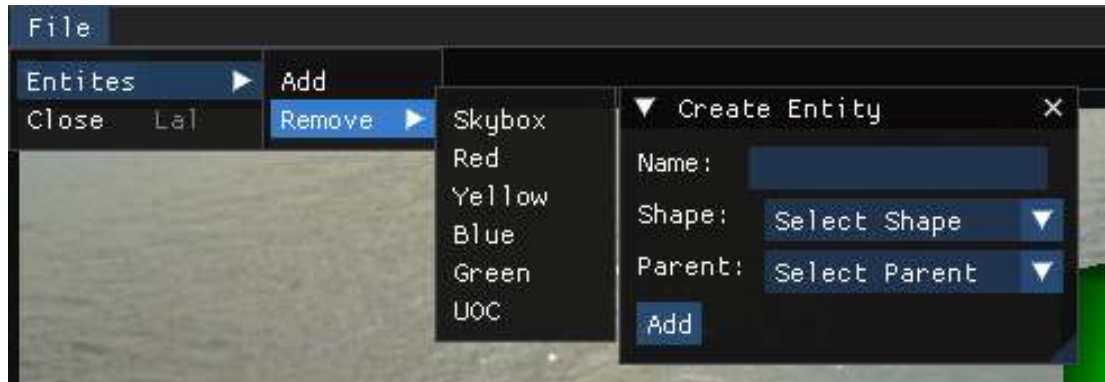


Figure 26: Extended View of Menu Bar with Entity Addition Window Open

5.1.2 Inner Workings

All necessary actions take place inside the `pyGLV/GUI/SceneWindow.py` (See *Section 5.3*). The menu bar functionality relies on the following functions:

- `mainMenuBar()`: creates and draws the menu bar on the window while also adding the options that are offered
- `showSelectableEntities(comp)`: displays the entities as options beginning from the component given as a parameter and continuing to its children. (See *Section 5.2*)

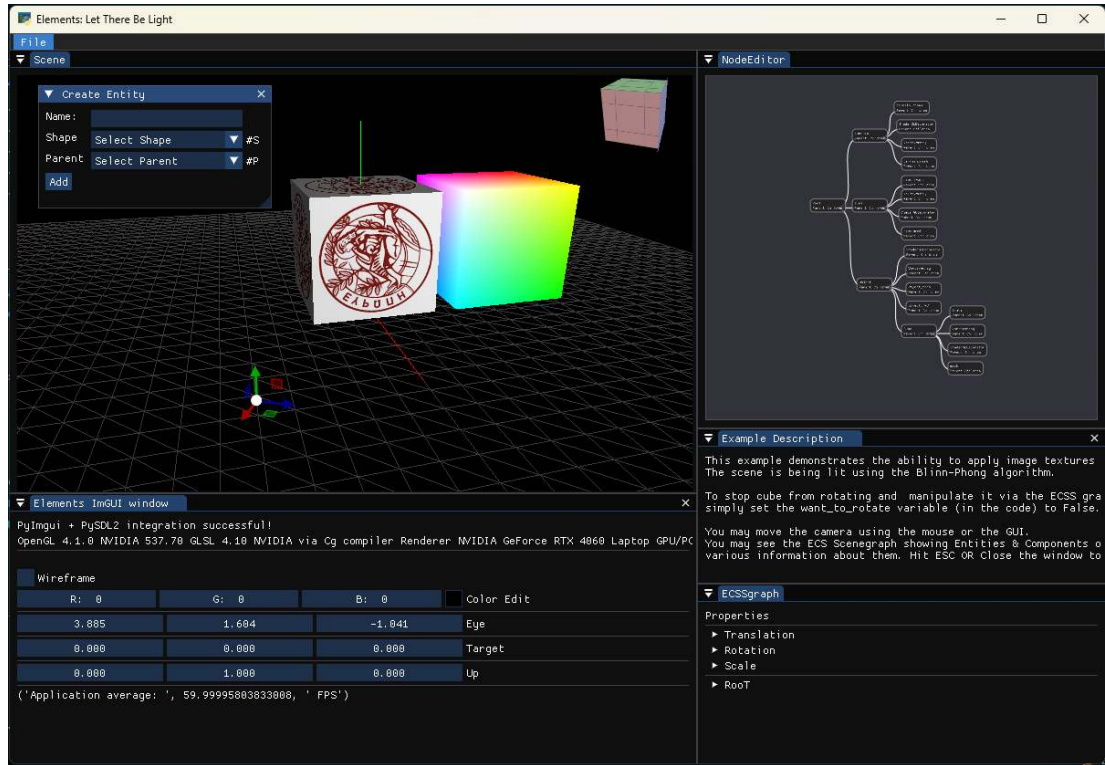


Figure 27: Window Menu Bar utilized in an example of entity addition

5.2 Entity Management

Until now, Elements was primarily static, lacking the capability to add or remove entities once an example was running. This process was restricted to code-based procedures. We sought to implement a method that would enable the dynamic creation of new entities during runtime, eliminating the need for 'hard-coding' entities prior to initiating the example.

5.2.1 Inner Workings

Entity addition leverages an existing module within the framework, **BasicShapes.py**. While it retains much of its original functionality, we have made some adjustments to enhance its integration and usability within the new system.

All the functions regarding the use of the above module take place inside the **pyGLV/GUI/SceneWindow.py** module, through the following functions which are responsible for displaying the appropriate windows (See Figure 28):

- **addEntityWindow()**: displays the window for the addition of a new entity

This uses the following attributes of the SceneWindow module:

- **selected_parent**: for keeping the parent that the entity will be appended to, if none, the entity will be a child of the root entity

- **shape**: for keeping the kind of shape the entity will have
- **name**: for keeping the name that will be given to the entity, if none, the entity will have a default name according to its shape

Note: Entities are added on the next loop after the add button has been clicked. The entity is kept in a `tmp` variable and a flag is turned to `True` to signal entity addition along with the `update_needed` flag to signal an update for the node editor

- `removeEntityWindow()`: displays a confirm dialog for the removal of the selected entity.

This uses the following attributes:

- **to_be_removed**: holds a reference to the entity we want to remove.

Note: The above features are accessible through the new Menu Bar on the top of the main window (5.1)

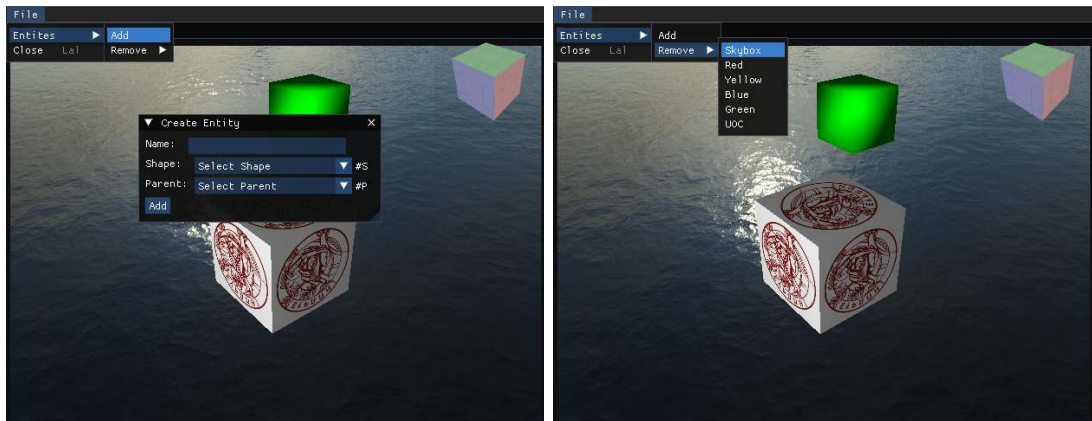


Figure 28: Entity Addition and Removal Windows

5.3 Scene Window Management

With all the added functionalities and features, we run into a problem. Due to the `pyGLV/GUI/ImguiDecorator.py` being the main module that does all the UI related features, all the added functionality was added to this very module.

This introduced complexity to the system due to the huge added weight to the module. It also made debugging and managing the system for maintenance or updates difficult.

To prevent this, all functionality related to the new type of UI, was moved into a new standalone module named `pyGLV/GUI/SceneWindow.py`. This not only made the framework more tidy but also more manageable and welcoming to maintenance and updates.

5.3.1 Inner Workings

The module it-self consists of the following attributes:

- **gizmo**: instance of pyGLV/GUI/Gizmos.py for the scene
- **changed**: checker variable indicating a change in the system
- **selected_parent**: holds the selected parent for the entity that is going to be added
- **shape**: holds the shape of the new entity as a string
- **add_entity**: checker variable to signal the addition of a new entity on the next loop cycle
- **remove_entity**: checker variable to signal the removal of an entity on the next loop cycle
- **entities**: list of all the entities that are present in the ECSS
- **to_be_removed**: reference to an entity that is going to be removed on the next loop cycle
- **_buffer**: reference to the instance of the framebuffer used to display the scene

and the following functions:

- **mainLoopWindow(view, wireframe, selected, currOperation)**: the main window loop function. Responsible for displaying all the windows of the main dockspace.
- **mainMenuBar()**: creates and draws the menu bar on the window while also adding the options that are offered
- **showSelectableEntities(comp)**: displays the entities as options beginning from the component given as a parameter and continuing to its children. This is for the remove entity functionality which will be discussed later, on Entity Management section
- **addEntityWindow()**: displays the window for the addition of a new entity
This uses the following attributes of the SceneWindow module:

- **selected_parent**: for keeping the parent that the entity will be appended to, if none, the entity will be a child of the root entity

- **shape**: for keeping the kind of shape the entity will have
- **name**: for keeping the name that will be given to the entity, if none, the entity will have a default name according to its shape

Note: Entities are added on the next loop after the add button has been clicked. The entity is kept in a tmp variable and a flag is turned to True to signal entity addition along with the `update_needed` flag to signal an update for the node editor

- **removeEntityWindow()**: displays a confirm dialog for the removal of the selected entity.

This uses the following attributes:

- **to_be_removed**: holds a reference to the entity we want to remove.
- **update_entities(component)**: Updates the ECSS when a new entity is added. Recursive function beginning from the given component.

6 Known Bugs

6.1 Camera Gizmo

While the camera gizmo functions effectively and operates smoothly, there is a minor bug related to the mathematical calculations used to derive the view matrix. Specifically, the horizontal rotation appears inverted; for instance, clicking on the right side of the gizmo correctly rotates the gizmo itself, but results in an opposite rotation in the actual scene.

Additionally, there is a slight issue that arises when the camera's target does not point to the center of the scene. This occurs because the gizmo generates a transformation (TRS) matrix for the camera instead of a view matrix. As a result, when extracting the eye position from that matrix, the target is not accounted for, leading to unexpected behavior.

6.2 Transformation Gizmo Alignment

While the gizmos successfully manipulate the selected object, they do not track the object accurately during transformations. This discrepancy arises from the gizmos operating within a different coordinate system, leading to variations in distance scaling between Elements and the gizmos. The primary issue manifests during the translation process (*See Figure 29*), as the rotation and scale functions perform correctly when not moved from the center of the system.

Additionally, the gizmos struggle to maintain alignment with the object due to the reversed horizontal axis in the framework's camera, which mirrors the behavior observed with the camera gizmo (*See Section 6.1*). This inconsistency further complicates the interaction between the gizmos and the objects they are intended to manipulate.

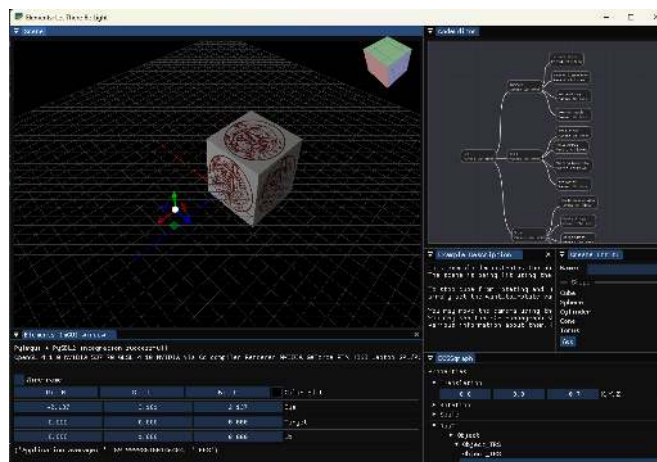


Figure 29: Example - Gizmo does not follow the object

7 Example

Having reviewed all the additional features we decide to add to Elements in order to ease its use and enhance its understand-ability towards new developers, let us have a look at a detailed example.

When the framework is first run, the developer is greeted with the Element's main screen (See Figure 30).

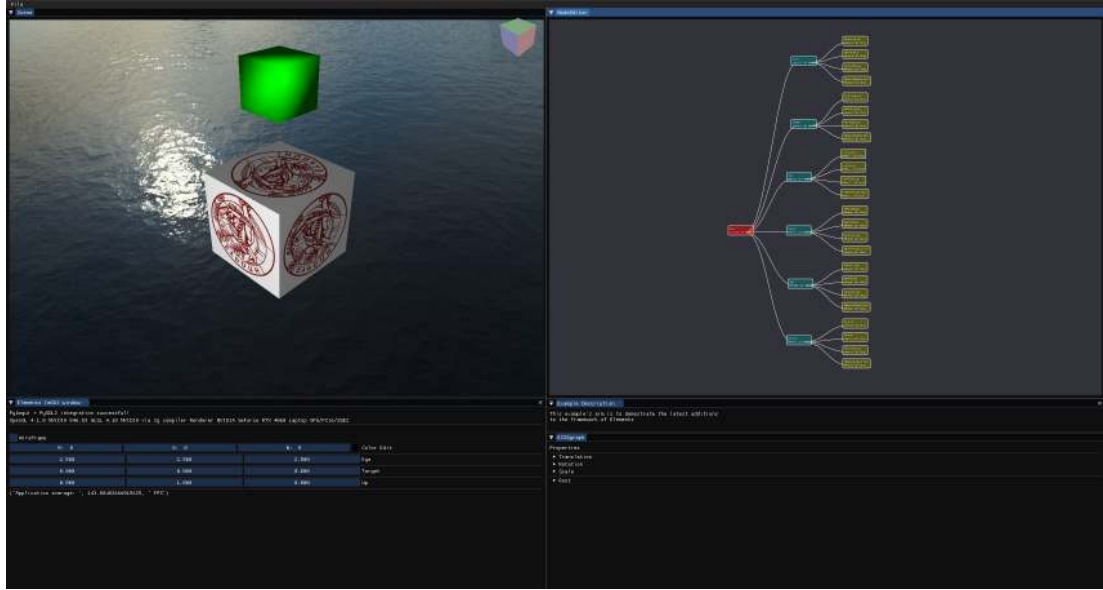


Figure 30: Elements Main Screen

7.1 Entity Addition

To add an entity:

- The developer begins by clicking on "File", located at the top-left corner of the main menu bar. From there, they hover over the "Add" option (*step 1 in Figure 31*), which triggers the entity addition menu to appear (*as shown in step 2 of Figure 31*).
- For this example, the entity will be named "TestCube," which the developer can enter into the designated field (*step 2 in Figure 31*). If this field is left empty, the entity will automatically be assigned a default name based on its selected shape. Next, the developer chooses one of the five available shapes, selecting "Cube" for this example (*step 3 in Figure 31*).
- The entity also requires a parent, which is selected in the subsequent step. If no parent is specified, the new entity will default to being a child of the root entity, as demonstrated in this example (*step 4 in Figure 31*).

- Finally, the developer clicks the "Add" button to complete the process (*step 5 in Figure 31*). Upon successful addition, a new set of nodes will appear in the node editor, confirming that the new entity has been integrated into the Entity-Component Scenegrph System (*step 6 in Figure 31*).

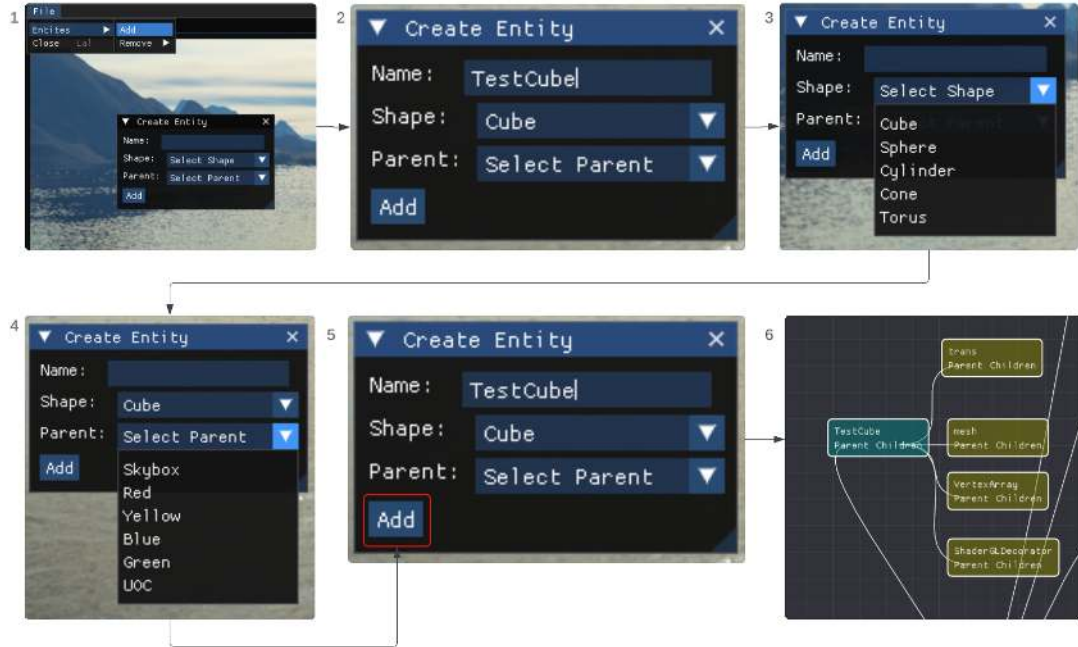


Figure 31: Entity Addition

7.2 Entity Manipulation

Having added a new entity, the developer can see the new nodes added into the scenegrph but not the entity itself, as in this example there is another entity already present from the start at the spawning position (*See Figure 32*)

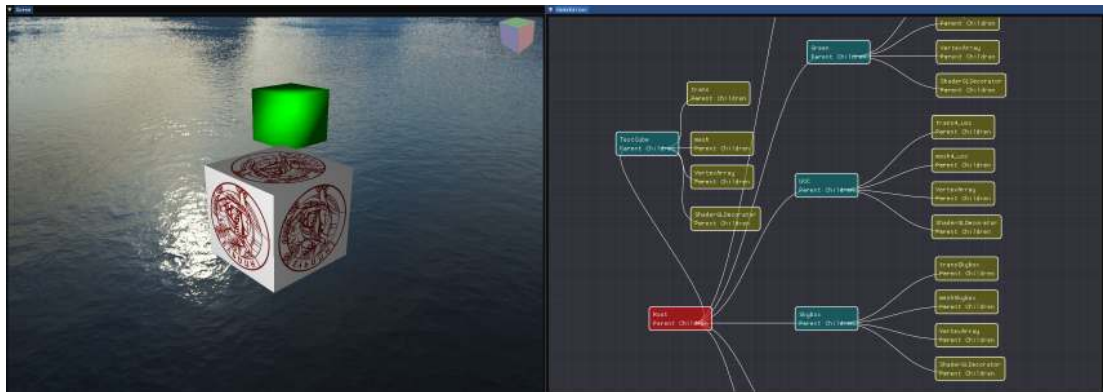


Figure 32: New entity not visible

To make the newly added entity visible, the developer must manipulate the existing entity that is currently obscuring it. The following steps outline this process:

- First, the developer selects the TRS (Translation, Rotation, Scale) Component of the entity they wish to manipulate—in this example, the "UOC" entity with the component named "trans4_uoc."
- Next, the developer chooses the type of transformation they wish to apply from the menu (*as shown in step 2 of Figure 33*). By default, the transformation selected is translation, so in this case, no additional selection steps are necessary after choosing the component. This action will cause the transformation gizmos to appear in the scene, allowing the developer to visually manipulate the entity (*step 3 in Figure 33*).
- To perform the translation, the developer uses the corresponding gizmo to drag the object in the desired direction. As the "UOC" entity is moved, the newly added entity will gradually become visible as it emerges from beneath the existing one (*step 4 in Figure 33*).

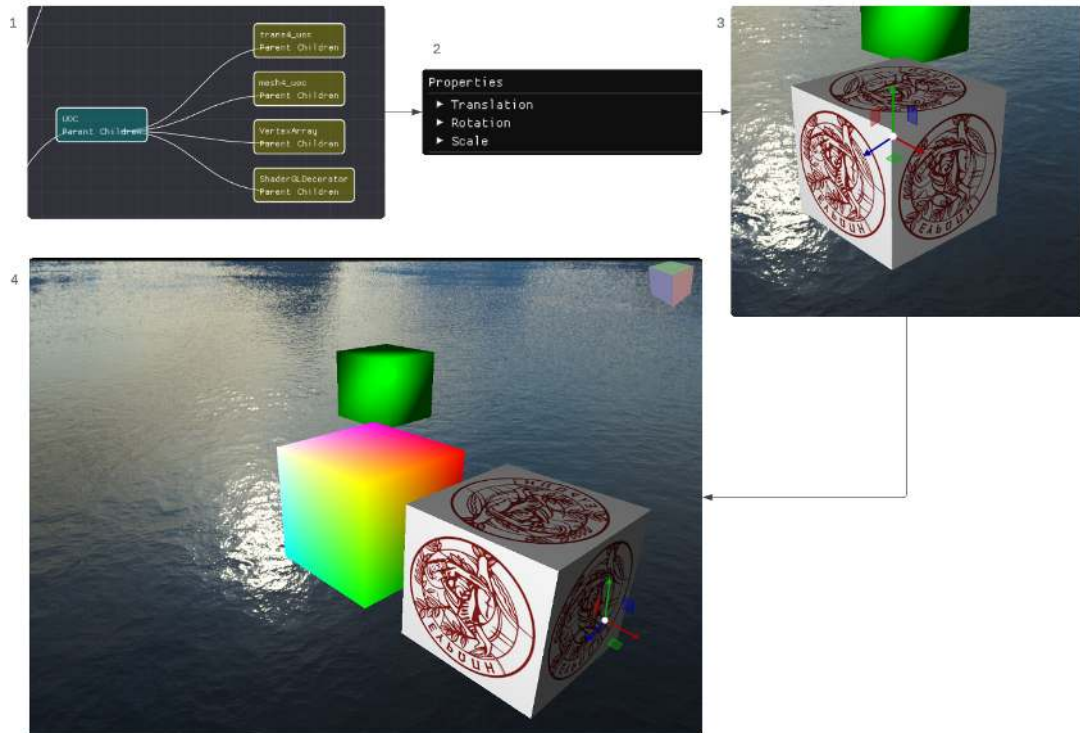


Figure 33: Entity Translation

To perform any other type of transformation, the process is as follows:

- The developer begins by selecting the entity they wish to manipulate, as described previously (See Figure 7.2). Next, they choose the type of transformation by expanding the appropriate node in the transformation window. This action will cause the corresponding gizmo to appear in the scene, providing a visual tool for manipulation.
- To rotate the entity, the developer selects the "Rotation" dropdown (*step 1 in Figure 34*), which will display the rotation gizmos in the scene (*step 2 in Figure 34*). The rotation is then performed by dragging the circles in the desired direction, effectively rotating the object along the chosen axis (*step 3 in Figure 34*).
- For scaling the entity, the developer selects the "Scale" dropdown (*step 4 in Figure 34*), which reveals the scale gizmos in the scene (*step 5 in Figure 34*). Scaling is achieved by dragging the scale arrows outward to increase the size or inward to reduce it, depending on the desired axis.

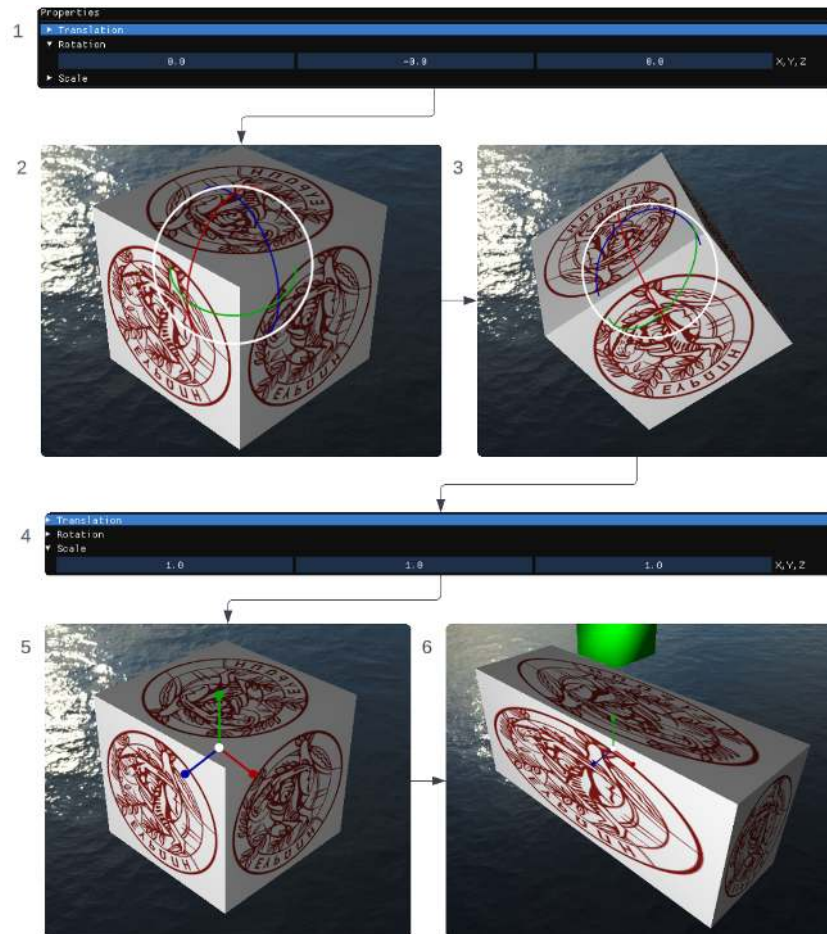


Figure 34: Entity Rotation and Scaling

7.3 Entity Re-Parenting

Another valuable feature introduced in this updated version of Elements is the ability to change an entity's parent without the need for complex, low-level coding. This task can now be accomplished entirely through the node editor, making it more accessible and intuitive for developers.

To re-parent an entity:

- The developer first navigates to the entity they wish to re-parent. In this example, the entity in question is the "UOC" Cube (*step 1 in Figure 35*).
- To begin the re-parenting process, the developer must delete the existing link between the entity and its current parent. This is done by selecting the link (*highlighted in red in step 1 of Figure 35*) and pressing the 'DELETE' key on the keyboard. Once the link is deleted, it will disappear from the node editor (*step 2 in Figure 35*).
- Next, to assign a new parent to the entity, the developer draws a new link connecting the two entities (*step 3 in Figure 35*). The order of the connection does not matter, as the parent is determined by which entity the child pin is connected to. In this example, the new parent will be the "Skybox."
- After the new link is drawn, it will permanently appear in the node editor (*step 4 in Figure 35*), and the change in hierarchy will be reflected in the ECSS Node Tree (*step 5 in Figure 35*).

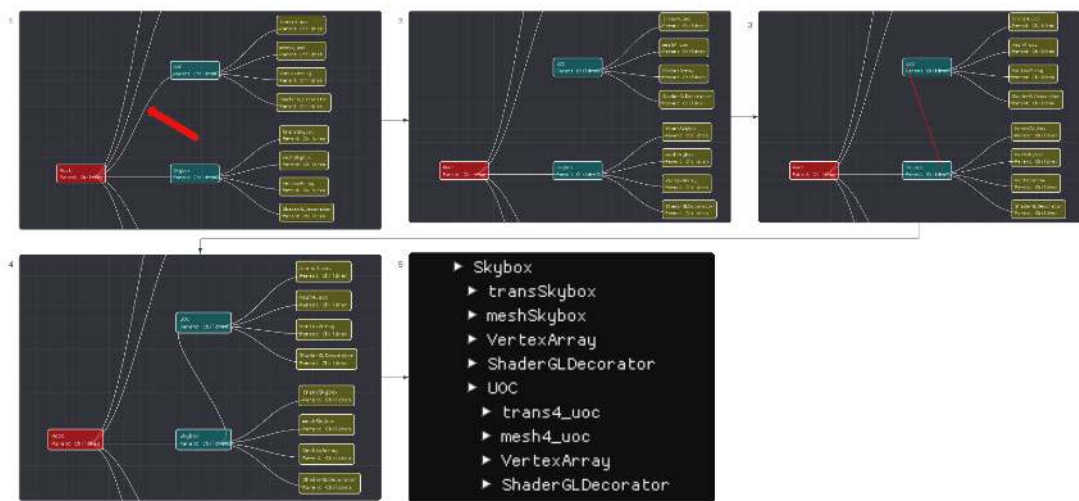


Figure 35: Entity Re-Parenting

7.4 Entity Deletion

To conclude this example, we will remove the entity that was added at the beginning using the new Menu Bar introduced in this updated version of Elements. The process for removing an entity is as follows:

- The developer starts by clicking on the "File" option, located at the top left corner of the main screen in the main menu bar. Next, they hover the mouse cursor over the "Entities" option and then over the "Remove" option. This action will reveal a list of all removable entities currently included in the ECSS (*as shown in step 1 of Figure 36*).
- The developer then selects the entity they wish to remove—in this example, "TestCube." Selecting this entity triggers a warning window (*step 2 in Figure 36*), prompting the developer to confirm the deletion to prevent accidental removals. Once the developer confirms the deletion, the changes are immediately reflected both in the scene, where "TestCube" is removed, and in the node editor, where the "TestCube" node and all its child nodes are deleted (*step 3 in Figure 36*).

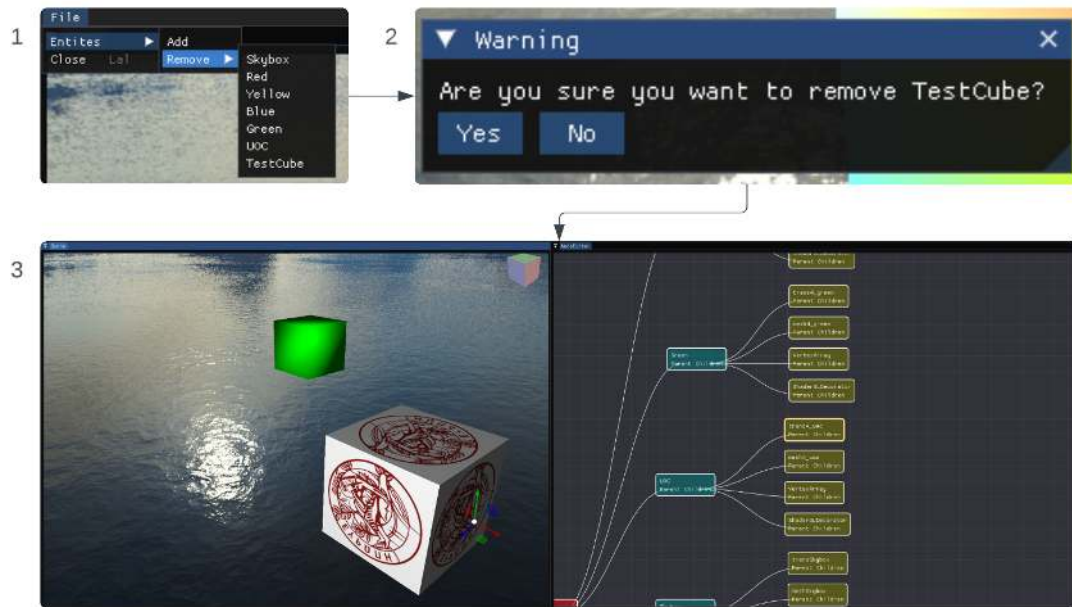


Figure 36: Entity Removal

8 Conclusion

In conclusion, the improvements made to Elements have resulted in a significantly more refined and user-friendly framework. By addressing critical areas such as entity transformation, camera manipulation, and the overall interaction workflow, we have aligned Elements more closely with the features and functionality expected from modern computer graphics frameworks. The integration of a gizmo system for direct manipulation, an enhanced node editor for visualizing and managing the scenegraph, and a more intuitive menu system for entity creation and deletion, all contribute to streamlining essential tasks that were previously cumbersome.

These enhancements make Elements more accessible, particularly for new users, by simplifying fundamental operations like transforming entities and controlling the camera view—core features that are integral to the workflow of all well-known game engines. With these changes, Elements now offers a much more versatile and approachable platform, making it a powerful tool for educational and development purposes. We believe these updates will not only improve the usability of the framework but also foster a deeper understanding of computer graphics principles by providing an open and transparent system for experimentation and learning.

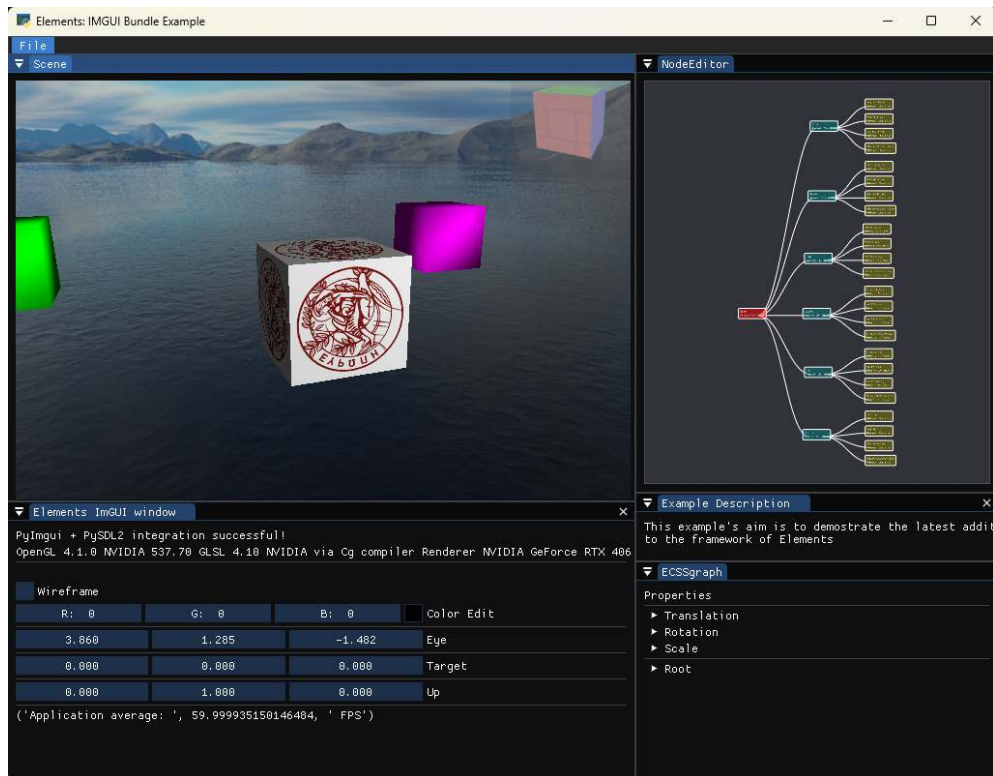


Figure 37: New Version of Elements

References

- [1] OpenGL. <https://www.opengl.org/>.
- [2] Simple DirectMedia Layer. <https://www.libsdl.org/>.
- [3] Omar Cornut. Dear ImGui. <https://github.com/ocornut/imgui>, 2022.
- [4] Papagiannakis George and Contributors. Elements framework. <https://github.com/papagiannakis/Elements>, 2024.
- [5] Cedric Guillemet. ImGuizmo. <https://github.com/CedricGuillemet/ImGuizmo>, 2022.
- [6] KinoxKlark. Cython-based Python Bindings for Dear ImGui. <https://github.com/pyimgui/pyimgui>, 2022.
- [7] pthom. ImGui Bundle. https://github.com/pthom/imgui_bundle, 2022.
- [8] thedmd. ImGui Node Editor. <https://github.com/thedmd/imgui-node-editor>, 2022.